

Week 8

K-Nearest Neighbors

*When Proximity Is
the Algorithm*

Section	Question it answers
1. From Parameters to Proximity	What if the algorithm had no parameters at all?
2. Testing KNN on All Four Tasks	Does nearest-neighbor classification work on our data?
3. What Distance Means	How does the choice of metric encode domain knowledge?
4. The Curse of Dimensionality	Why does KNN break in high dimensions?
5. Choosing k	How does the number of neighbors control bias and variance?
6. Computational Cost	When does KNN become impractical?
7. Bringing It Together	Where does KNN fit in the comparison table?

1. From Parameters to Proximity

What if the algorithm had no parameters at all?

Every algorithm we have studied so far learns something from training data and stores it as parameters. Logistic regression learns a weight vector — seven numbers for EMG binary classification (six weights plus an intercept). Naive Bayes learns means and variances — twelve numbers for the same task. These parameters compress the training data into a compact model that generalizes to new observations.

K-nearest neighbors (KNN) takes a fundamentally different approach: it stores the entire training set and makes no attempt to summarize it. When a new trial arrives, KNN computes the distance from that trial to every training trial, finds the k closest ones, and takes a majority vote among their labels. There is no training phase, no learned parameters, no model at all — just data and a distance function.

The algorithm in four steps

- **Store** — Save all training trials and their labels. This is the entire “training” phase.
- **Compute distances** — For a new trial, measure the distance to every stored training trial.
- **Find neighbors** — Select the k training trials with the smallest distances.
- **Vote** — Assign the new trial the label that appears most often among its k neighbors.

This is called **lazy learning** — all computation is deferred to prediction time. The model has a single hyperparameter: k , the number of neighbors to consult. As we will see, this one number controls the bias-variance tradeoff in a way that is both intuitive and visual.

Figure 1 illustrates this with a simulated clinical screening dataset where patients are measured on grip strength and reaction time, and the two groups (healthy vs impaired) overlap in a nonlinear pattern. With $k = 1$, the boundary is jagged and full of small islands — every patient, including noisy outliers, gets their own region. Training accuracy is 100% because the model memorizes the data, but it would generalize poorly to new patients. With $k = 11$, the boundary smooths into a clean curve that captures the true separation between groups. With $k = 201$, the boundary collapses to a near-straight line — the model is too simple to capture the nonlinear structure and training accuracy drops to 82%.

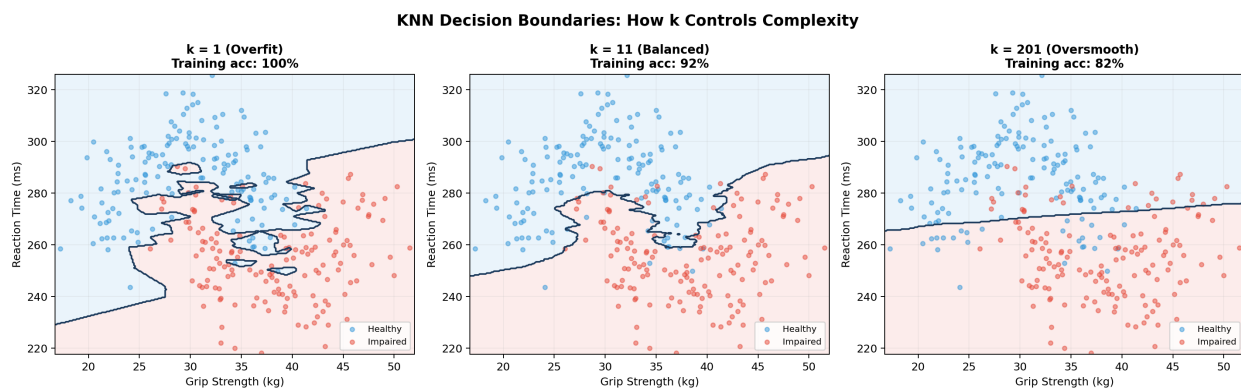


Figure 1. KNN decision boundaries for a simulated clinical screening (grip strength vs reaction time).

Left: $k=1$ overfits — memorizes every patient including noise (100% train accuracy). Center: $k=11$ captures the true nonlinear boundary between groups (92%). Right: $k=201$ underfits — the near-linear boundary cannot capture the curved separation (82%).

Evaluation with leave-one-subject-out

In our motor control dataset, each subject contributes 24 trials. To evaluate KNN fairly, we use the same leave-one-subject-out (LOSO) protocol from previous weeks: hold out all trials from one subject, store the remaining 19 subjects' trials as the training set, then classify each held-out trial by finding its k nearest neighbors among the stored trials. The held-out subject's data is never part of the stored training set, so 100% training accuracy (as with $k = 1$) does not guarantee good LOSO performance. Overfitting means KNN memorizes quirks of the training subjects that do not generalize to the held-out subject.

2. Testing KNN on All Four Tasks

Visualizing the neural data

We will test KNN on the same dataset from Week 6: firing rates from 80 simulated cortical neurons recorded during a center-out reaching task with 8 directions, across 20 subjects (10 healthy, 10 motor-impaired). Each trial is an 80-dimensional vector — one firing rate per neuron. To visualize what KNN will be working with, we apply PCA to reduce these 80 dimensions to two.

Figure 2a shows the result. Each dot is one trial, colored by the true reach direction. The eight direction clusters arrange themselves in a **ring** in PCA space. This is not a coincidence — it reflects the cosine tuning of the underlying neurons. PC1 and PC2 each capture about 36% of the variance (Figure 2b), nearly equal because they represent the cosine and sine components of the population vector. Together, just two PCs capture 72% of the variance across 80 neurons. This is the low-dimensional structure that KNN's distance computation will exploit.

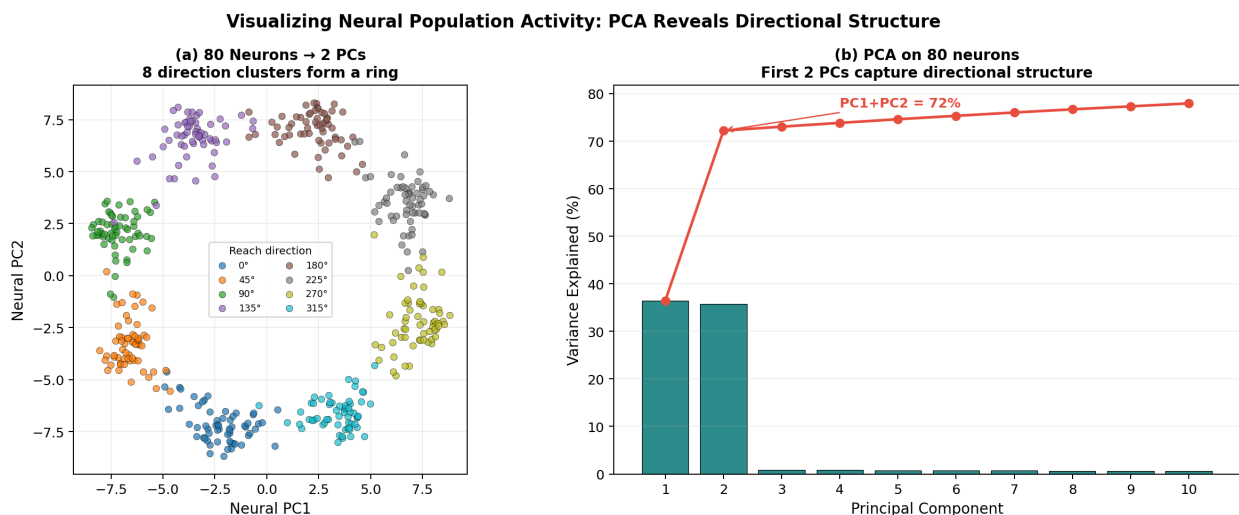


Figure 2. PCA of 80-neuron firing rates from our Week 6 dataset. (a) Trials projected onto the first two PCs, colored by reach direction. The eight clusters form a ring because cosine-tuned neurons produce sinusoidal population activity. (b) Variance explained: PC1 and PC2 each capture ~36% (cosine and sine components), together accounting for 72% of total variance.

KNN boundaries in this space

Now we apply KNN to classify reach direction from these neural features. Figure 3 shows the decision boundaries in the same PCA space at three values of k . With $k = 1$, the boundaries are jagged with small pockets — overfitting to individual trials. With $k = 5$, the regions become smooth and each direction claims a coherent wedge of the ring. With $k = 101$, the regions are very broad and some directions start to lose territory to their neighbors.

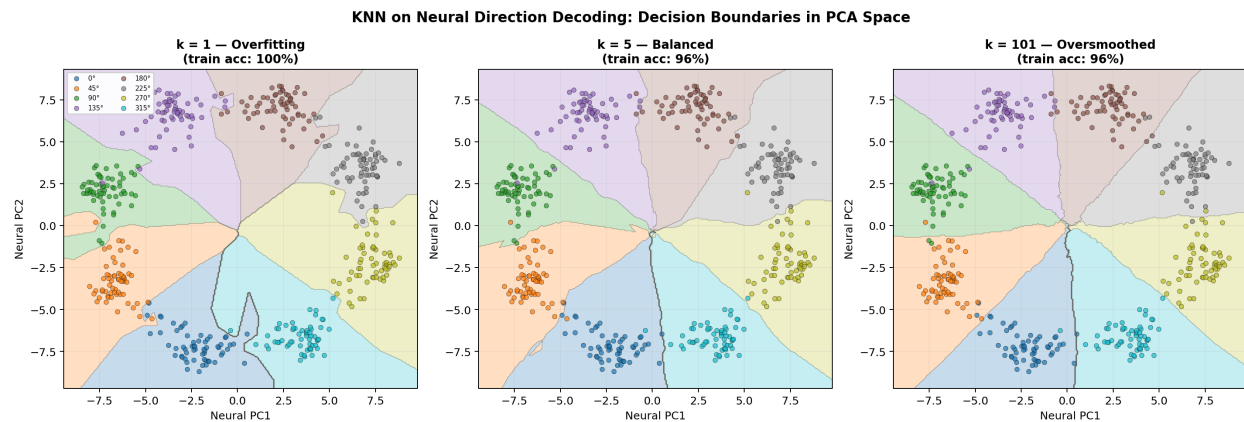


Figure 3. KNN decision boundaries for neural direction decoding in PCA space (80 neurons projected to 2 PCs). Left: $k=1$ overfits with jagged boundaries (100% train accuracy). Center: $k=5$ produces smooth regions that respect the circular arrangement of direction clusters (96%). Right: $k=101$ oversmooths with broad regions (96% train, but begins to misclassify boundary trials).

Results across all four tasks

Following the same protocol as Weeks 5–7, we test KNN ($k = 5$, Euclidean distance after standard scaling) on all four task/feature combinations using leave-one-subject-out cross-validation. Figure 4 shows the results alongside logistic regression and Naive Bayes for comparison.

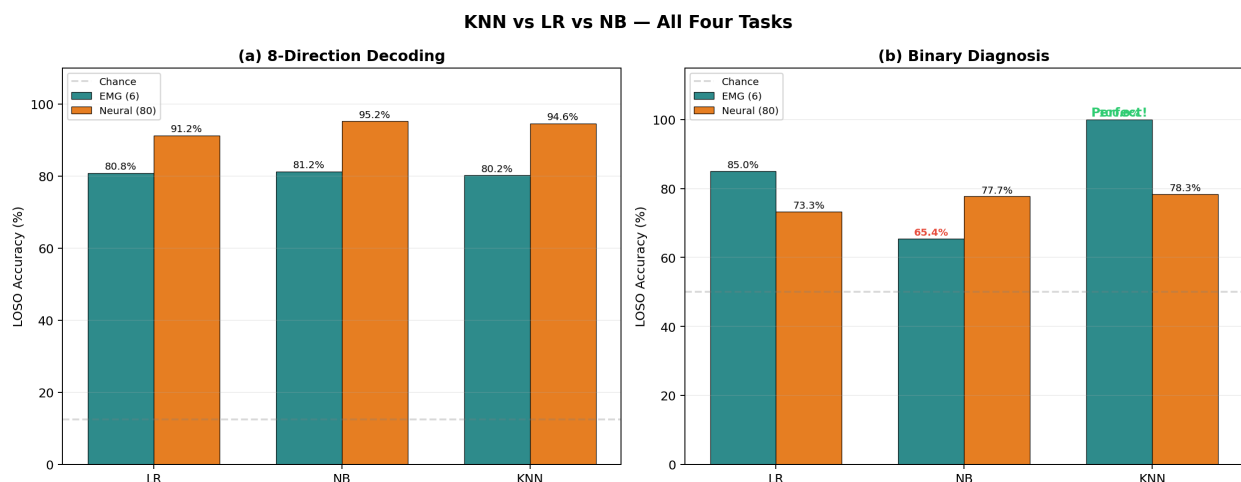


Figure 4. LOSO accuracy across all three methods and both feature sets. Left: 8-direction decoding. KNN is competitive on both feature sets. Right: binary diagnosis. KNN achieves 100% on EMG — a perfect result that no previous method has matched. NB's 65.4% EMG catastrophe from Week 7 is shown for contrast.

Two surprises in the results

The first surprise is **EMG binary diagnosis: 100%**. Every LOSO fold gets every trial correct. This is not overfitting — this is leave-one-subject-out, the same rigorous protocol we have used since Week 5. KNN beats logistic regression (85.0%), Naive Bayes (65.4%), and even PCA-rescued NB (85.0%). The second surprise is that **neural direction decoding (94.6%)** is competitive with Naive Bayes (95.2%) and close to the population vector (95.6%) despite KNN having no model of cosine tuning curves at all.

Why 100% on EMG binary?

Each subject contributes 24 trials, all of the same class (either all healthy or all impaired). In LOSO, the test set is one subject's 24 trials. KNN finds the nearest training trials — which, for EMG data, come from the most similar *subjects*. Because healthy and impaired subjects form two well-separated clusters in the 6-dimensional EMG space, the nearest neighbors always belong to the correct group. Figure 5 makes this visible.

This is a conceptually different kind of classification from what LR and NB do. Those models learn a *rule* (a weight vector or a set of class-conditional distributions) and apply it to each trial independently. KNN asks a different question entirely: **“who does this subject look like?”** This is actually the question clinicians ask implicitly when they compare a new patient's movement patterns to those of previous patients they have seen.

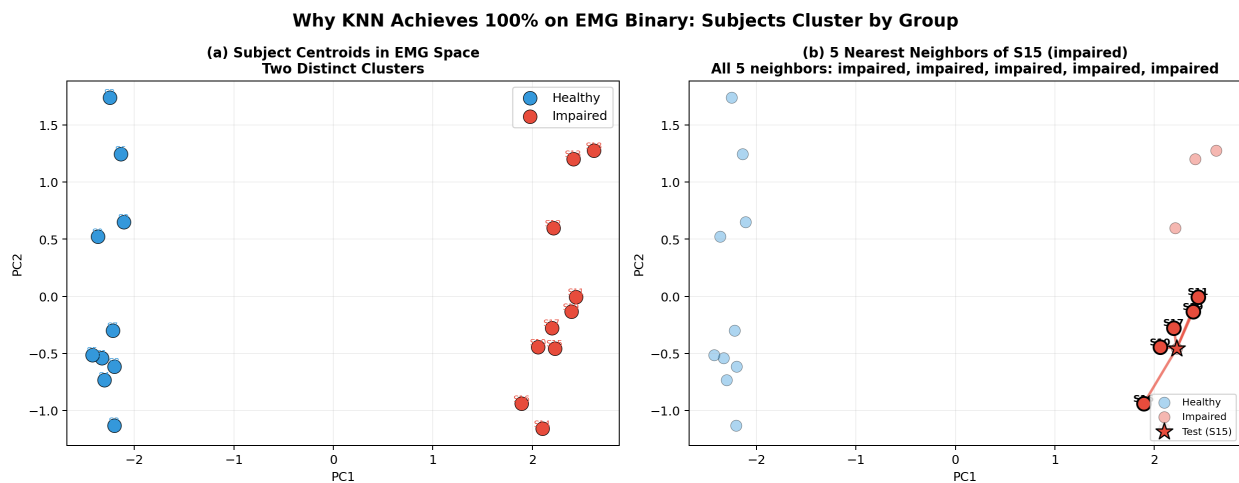


Figure 5. Why KNN achieves perfect accuracy on EMG binary. (a) Subject centroids in PCA space form two distinct clusters. (b) For a test subject (S15, impaired), all 5 nearest neighbors are impaired — the vote is unanimous. This holds for every subject in every fold.

But if KNN is this effective, why not use it for everything? The next three sections reveal its three fundamental limitations: sensitivity to distance metrics, vulnerability to high dimensions, and computational cost at scale.

Is the 100% result specific to EMG?

The perfect EMG binary result raises a question: is this a property of KNN, or a property of *what we measured*? To find out, we add a third feature set: joint angles (shoulder and elbow) computed from a two-link planar arm model using inverse kinematics for each reaching trial. Both healthy and impaired subjects reach to the same eight targets, so their joint configurations should be nearly identical — the impairment changes *how* muscles are recruited, not *where* the hand goes.

Figure 6 confirms this reasoning. Joint angles achieve 99.2% on direction decoding — near-perfect, because a two-joint planar arm has no redundancy: two joint angles uniquely determine the (x, y) hand position, and vice versa. Each target maps to exactly one joint configuration. The small error (0.8%) comes from endpoint variability and subject-level differences in anthropometry. But on binary diagnosis, joint angles score 42.9%, **below chance**. The kinematics carry no information about impairment because both groups reach to the same targets with the same posture.

Note that this near-perfect direction result depends on the arm having exactly two degrees of freedom for a two-dimensional task. A three-joint arm (shoulder, elbow, and wrist) reaching to the same targets would be *kinematically redundant* — infinitely many joint configurations produce the same hand position. Different subjects would adopt different strategies for distributing motion across joints, adding inter-subject variability that would reduce LOSO accuracy. Our two-joint model eliminates this redundancy, making joint angles a near-perfect predictor of direction but a useless predictor of impairment.

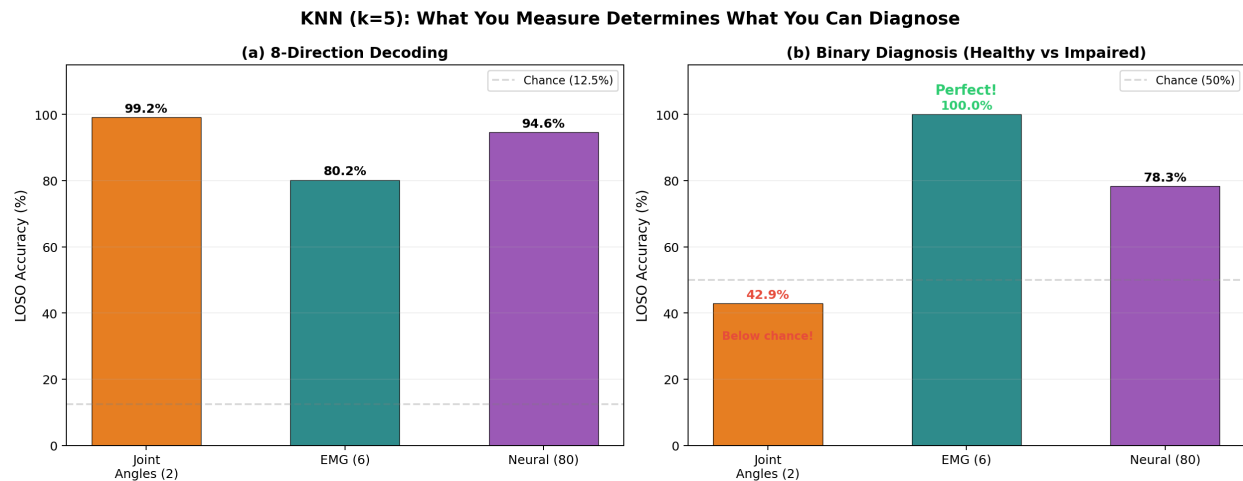


Figure 6. KNN ($k=5$) across three feature sets. Direction decoding (left): joint angles achieve 99.2% because the two-joint arm has no redundancy — each target maps to a unique posture. Binary diagnosis (right): joint angles are below chance because both groups reach to the same targets. EMG is perfect because impairment changes muscle recruitment strategy, not movement endpoint.

This is a fundamental lesson for motor control research: **the choice of what you measure determines what you can decode.** Kinematics tells you where the hand went. EMG tells you how the nervous system coordinated the muscles to get it there. Neural activity encodes both. KNN reveals this structure because it classifies by similarity — and what counts as “similar” depends entirely on which features you give it.

3. What Distance Means

The metric encodes your domain knowledge

KNN has exactly one assumption: **similar inputs should have similar outputs**. Everything depends on what “similar” means, and that depends entirely on the distance metric. Different metrics embed different assumptions about the data, and the right choice can make a substantial difference.

Euclidean distance measures straight-line distance in feature space. After standard scaling, it treats all features equally and is sensitive to both the direction and amplitude of the feature vector. This is the default for most KNN implementations.

Manhattan distance sums the absolute differences along each axis. It is more robust to outliers than Euclidean distance because a single extreme value contributes linearly rather than quadratically.

Cosine distance measures the angle between two feature vectors, ignoring their magnitude entirely. Two trials that point in the same direction in feature space are “close” even if one has twice the amplitude of the other. For direction decoding, this is exactly right: the neural population activity pattern for a 90° reach has a characteristic *shape* regardless of trial-to-trial variation in overall firing rates.

Figure 7a confirms this. On neural direction decoding, cosine distance (95.8%) outperforms Euclidean (94.6%), and on EMG direction, cosine reaches 84.6% compared to 80.2% for Euclidean. Figure 7b illustrates why: two trials from the same reach direction but with different amplitudes have a cosine distance of zero (they point the same way) but a large Euclidean distance (their endpoints are far apart).

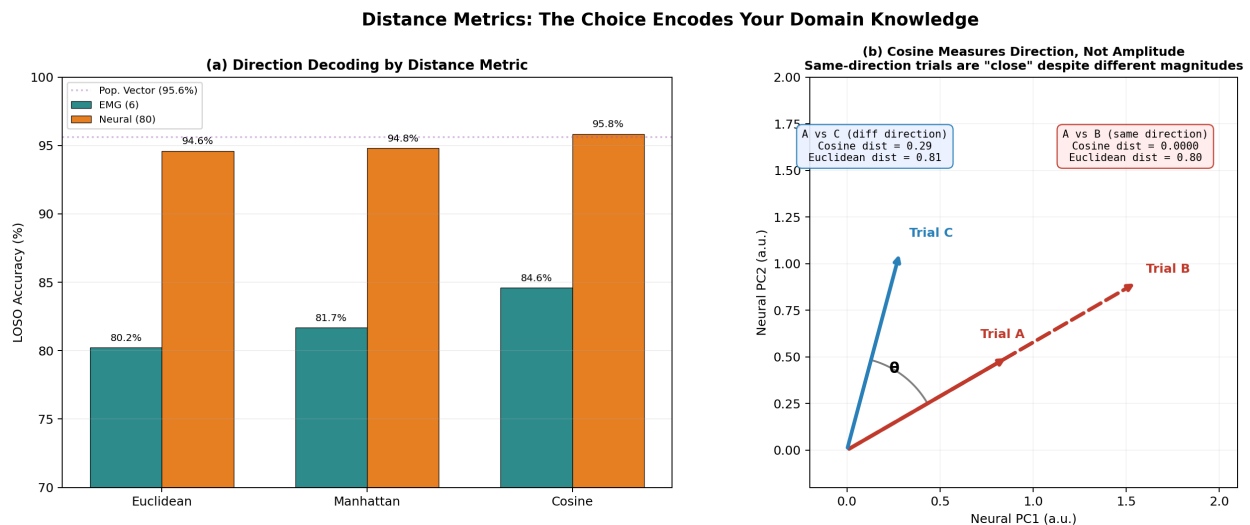


Figure 7. (a) Direction decoding accuracy by distance metric. Cosine distance wins on neural data because direction is encoded in the pattern shape, not its amplitude. The dashed line marks the population vector result from Week 6. (b) Why cosine works: Trials A and B represent the same reach direction but with different overall firing rates — cosine sees them as identical (distance = 0) while Euclidean sees them as far apart (0.80). Trial C is a different direction: cosine correctly flags the large distance (0.29).

A hidden connection: KNN and the population vector

This result reveals a deep connection between two seemingly different methods. Recall from Week 6 that the population vector decoder works by computing a mean activity pattern for each direction from the training data, then classifying a new trial by finding which mean pattern it most closely matches in angle. Cosine distance measures exactly this — the angle between two vectors, ignoring their length. So when KNN with cosine distance looks for the k nearest training trials, it is finding the trials whose neural activity patterns point in the most similar direction in 80-dimensional space.

Now consider what happens as k grows. With $k = 5$, KNN takes a majority vote among 5 nearby trials. With $k = \text{all}$ (every training trial votes), the vote is dominated by whichever direction has the most trials pointing in a similar direction to the test trial — which is exactly what the population vector does when it assigns the test trial to the direction whose mean pattern has the smallest cosine angle. In other words, the population vector decoder is a special case of KNN: it uses cosine distance with k equal to the entire training set. Both achieve 95.6% on our data because they are performing the same underlying computation — matching the direction of neural activity patterns.

Figure 8 confirms this equivalence empirically. Panel (a) compares predictions trial by trial: 479 out of 480 trials receive identical predictions from both methods. Panel (b) shows why — for a single test trial, the cosine similarity profile across the eight directions is nearly identical whether computed against the direction means (population vector) or averaged across individual training trials (KNN). Panel (c) shows the mathematical connection: the cosine similarity to a direction's mean is approximately the average cosine similarity to that direction's individual trials. Panel (d) traces this convergence: at $k = 1$, KNN cosine agrees with PopVec on 95.6% of trials (averaged across all 20 LOSO folds). As k increases, agreement climbs steadily and reaches exactly 100% when every training trial votes.

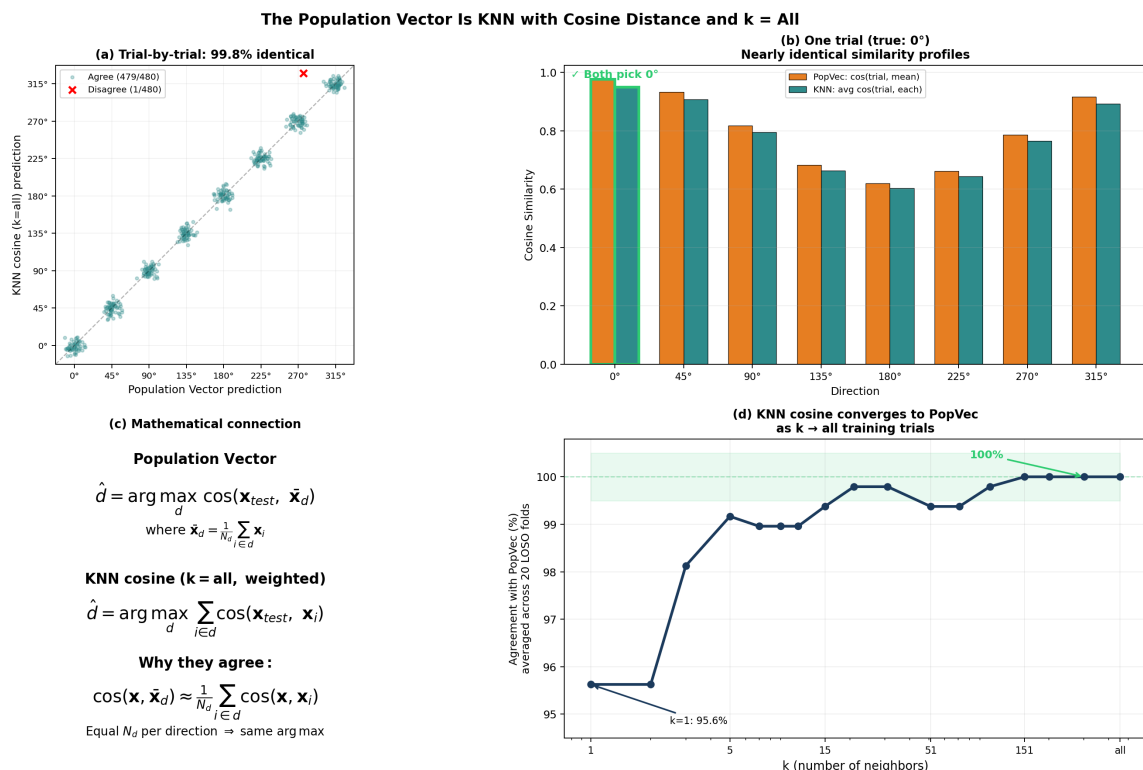


Figure 8. Empirical demonstration that the population vector decoder is a special case of KNN with cosine distance. (a) Trial-by-trial predictions agree on 479/480 trials. (b) Similarity profiles are nearly identical for a single trial. (c) The mathematical connection: cosine to the mean approximates the mean of cosines. (d) Agreement averaged across 20 LOSO folds: convergence from 95.6% at $k=1$ to 100% when all training trials vote.

Mahalanobis distance accounts for correlations between features by weighting each dimension by the inverse of its variance. Through PCA, we can see exactly what this means: project data onto principal components, and Mahalanobis distance becomes the sum of squared PC differences each divided by the corresponding eigenvalue (Figure 9d). PCA + Euclidean computes the same sum *without* dividing by the eigenvalue — it treats all PCs equally. These two metrics are related but **not equivalent**, and the difference matters enormously.

In theory, the $1/\lambda$ weighting should help: it normalizes each dimension by how much it varies, preventing high-variance features from dominating the distance. In practice, it fails catastrophically on our 80-neuron data (Figure 9c). The reason is visible in Figure 9a: PCs 1-2 have large eigenvalues (they carry the direction signal), while PCs 3-80 have near-zero eigenvalues (noise). When Mahalanobis divides by a tiny eigenvalue, a small random fluctuation on that PC gets amplified into an enormous distance contribution (Figure 9b). With each noise PC you add, another large random term enters the sum, progressively burying the signal from PCs 1-2. This is why Mahalanobis accuracy drops steadily: 95% at PCA(2), 62% at PCA(10), 33% at PCA(80). Every additional noise dimension makes classification worse.

PCA + Euclidean avoids this collapse. Without the $1/\lambda$ weighting, each noise PC contributes a small random amount to the distance rather than an enormous one. The noise washes out instead of accumulating, so accuracy remains at ~95% regardless of how many PCs are included.

The takeaway: **truncated PCA + Euclidean acts as a regularized form of Mahalanobis**. Throwing away the small-eigenvalue PCs prevents the distance from being corrupted by amplified noise. Even keeping all PCs is safe with Euclidean distance because it never inverts the eigenvalues. This connects directly to Week 7: PCA is not just dimensionality reduction — it is implicit regularization.

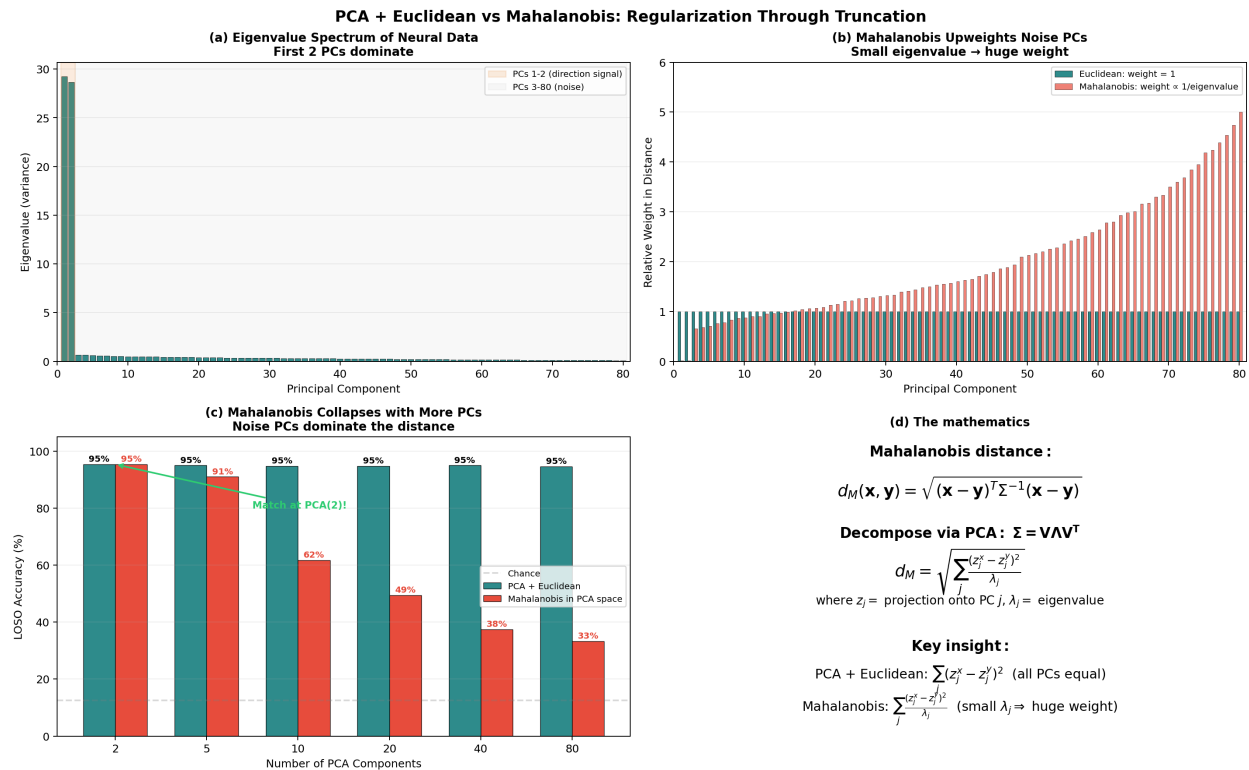


Figure 9. PCA + Euclidean vs Mahalanobis distance. (a) Eigenvalue spectrum: PCs 1-2 carry direction signal, PCs 3-80 are noise. (b) Mahalanobis weights each PC by 1/eigenvalue, amplifying noise dimensions. (c) At PCA(2) both methods achieve 95%; as more PCs are added, Mahalanobis collapses to 33% while PCA+Euclidean stays at 95%. (d) The mathematical relationship between the two distance metrics.

The broader lesson is that the distance metric is not a technical detail — it is a **modeling decision** that encodes what you believe matters about your data. Choosing cosine says “direction matters, amplitude does not.” Choosing Mahalanobis says “normalize by variance” — but beware: with noisy high-dimensional data, this amplifies the very dimensions you should ignore. Choosing PCA + Euclidean says “keep the signal, discard the noise, then treat what remains equally.” The metric is where domain knowledge enters KNN.

4. The Curse of Dimensionality

In high dimensions, nothing is near anything

KNN relies on a simple intuition: nearby points should have similar labels. But in high-dimensional spaces, this intuition breaks down because the concept of “nearness” itself becomes meaningless. As the number of features grows, all points become approximately equidistant from each other. The nearest neighbor is barely closer than the farthest point, and the majority vote becomes essentially random.

We can demonstrate this directly with our data. Take the 6 real EMG features, which give KNN 80.2% accuracy on direction decoding. Now add random noise features — features that carry zero information about direction. The signal does not change; the real 6 muscles are still there. But watch what happens to KNN:

- 6 real + 0 noise = 80.2%
- 6 real + 10 noise = 60%
- 6 real + 100 noise = 37%
- 6 real + 1000 noise = 20% (near chance at 12.5%)

Adding noise features does not remove information — the 6 real muscles are still present. But in 1006 dimensions, the distances are dominated by the 1000 noise dimensions. The real signal is drowned out, and the “nearest” neighbor in 1006D is no more informative than a randomly chosen training point.

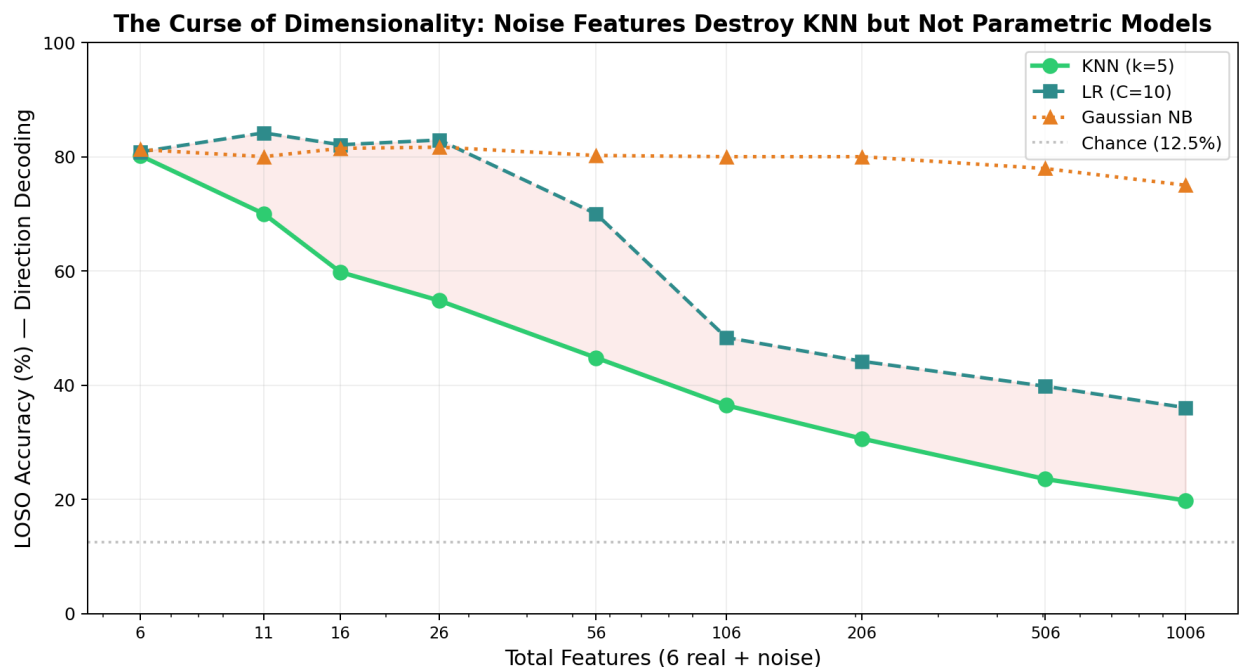


Figure 10. The curse of dimensionality. As noise features are added (x-axis, log scale), KNN accuracy collapses. LR degrades more slowly because regularization can shrink noise weights toward zero. NB is nearly immune because noise features produce flat class-conditional distributions that contribute no evidence to the posterior.

Why parametric models survive

Figure 10 shows that LR and NB handle the same noisy data far better than KNN. The reason is that parametric models have mechanisms to ignore irrelevant features. Logistic regression with regularization ($C=10$) pushes noise weights toward zero — it learns which features matter and which do not. Naive Bayes computes class-conditional distributions for each feature independently. For noise features, the distributions are identical across classes, so they contribute no evidence to the posterior and effectively cancel out. KNN has no such mechanism. It treats every feature equally when computing distance, so 1000 noise dimensions overwhelm 6 real ones.

What this means for neural data

Our neural dataset has 80 features — is that too many for KNN? Figure 11 shows the effect of applying PCA to reduce dimensionality before KNN. The result is instructive: PCA(2) + KNN achieves 95.4% on neural direction decoding, slightly *better* than raw KNN at 94.6%. The improvement is modest because 80 neurons with diverse preferred directions do not suffer badly from the curse — each neuron genuinely carries directional information. But with 200 or more neurons, which is increasingly common in modern recording technologies, dimensionality reduction becomes essential for distance-based methods.

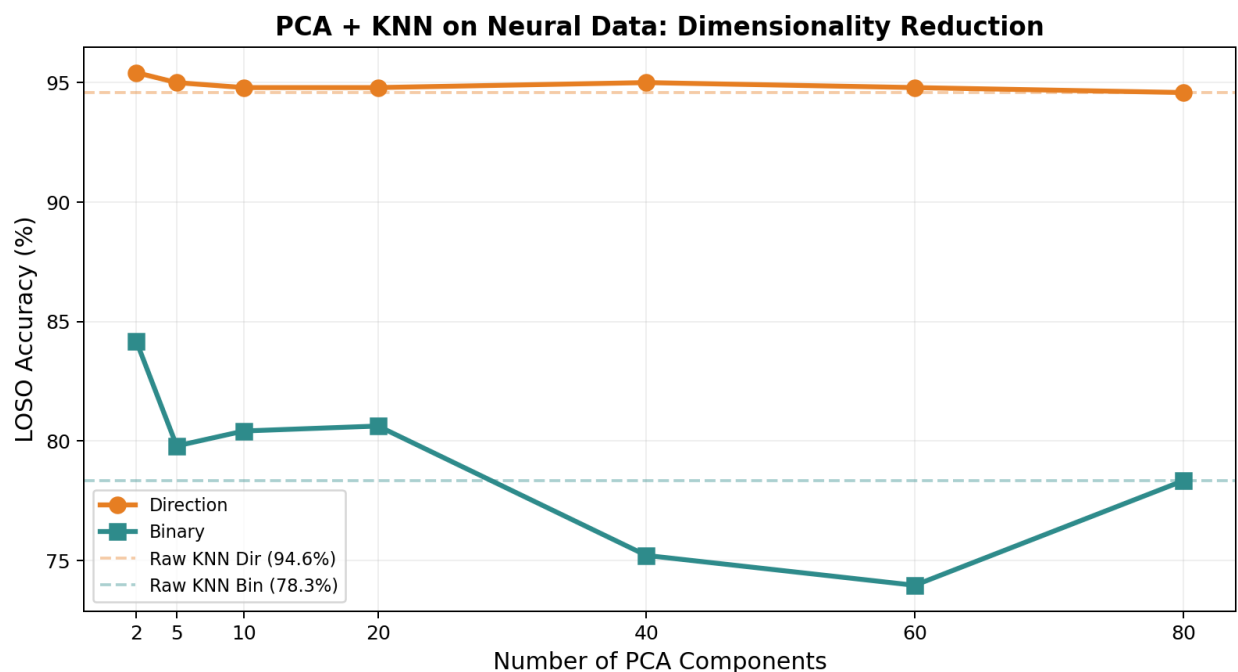


Figure 11. PCA + KNN on neural data. For direction decoding, even PCA(2) achieves 95.4%, slightly above raw KNN (94.6%). For binary diagnosis, performance peaks around PCA(5-10), suggesting the diagnostic signal lives in a low-dimensional subspace of neural activity.

This has practical implications for motor control research. Clean data — such as well-isolated single neurons recorded in a controlled lab setting, or EMG from surface electrodes over large muscles — tends to have high signal-to-noise ratios and relatively few truly informative dimensions. KNN works well here even without dimensionality reduction, because most features carry real information. Noisy data — such as high-density electrode arrays with many correlated or poorly isolated channels, or wearable sensors in uncontrolled clinical settings — is the opposite: many recorded dimensions, but the signal lives in a small subspace buried under noise.

In these settings, raw KNN will struggle because the noise dimensions dominate the distance computation. PCA (or other dimensionality reduction) becomes essential: it extracts the signal subspace and discards the noise before KNN ever computes a distance. The noisier your recording, the more you need to reduce dimensions first.

5. Choosing k — Bias-Variance in Action

The number of neighbors k is KNN's single hyperparameter, and it controls the bias-variance tradeoff in a way that is both intuitive and directly observable.

With $k = 1$, each test trial is classified by its single nearest neighbor. The decision boundary passes through the exact midpoint between neighboring training points. This model has **zero bias** — it can represent arbitrarily complex boundaries — but **maximum variance**, because a single noisy training trial can shift the prediction. With $k = N$ (the entire training set), KNN simply predicts the majority class for every trial: maximum bias, zero variance. The optimal k lies somewhere between these extremes.

Figure 12 sweeps k from 1 to 31 across all four tasks. Several patterns emerge. EMG binary diagnosis is 100% for every value of k — the healthy and impaired clusters are so far apart in EMG space that even with 31 neighbors voting, every neighbor still belongs to the correct group. Neural direction decoding improves from 91% at $k = 1$ to 95.6% around $k = 7-11$ as averaging over more neighbors smooths out trial-to-trial variability. Neural binary diagnosis peaks at $k = 5$ (78.3%) then declines, because the diagnostic signal is subtle and too many neighbors dilute it.

What about distance-weighted voting? A common refinement is to weight each neighbor's vote by $1/d$, so closer neighbors count more. In our data, this makes virtually no difference: uniform and weighted KNN produce identical results at every k . The reason is that our clusters are well separated. When all k neighbors belong to the same class — as in EMG binary — weighting a unanimous vote changes nothing. When the correct class dominates among neighbors — as in neural direction — the few wrong-class neighbors are already outvoted. Distance weighting matters most when classes overlap and the k neighbors contain a genuine mix of classes at similar distances, which is more common in noisy clinical data than in controlled lab recordings or simulated data.

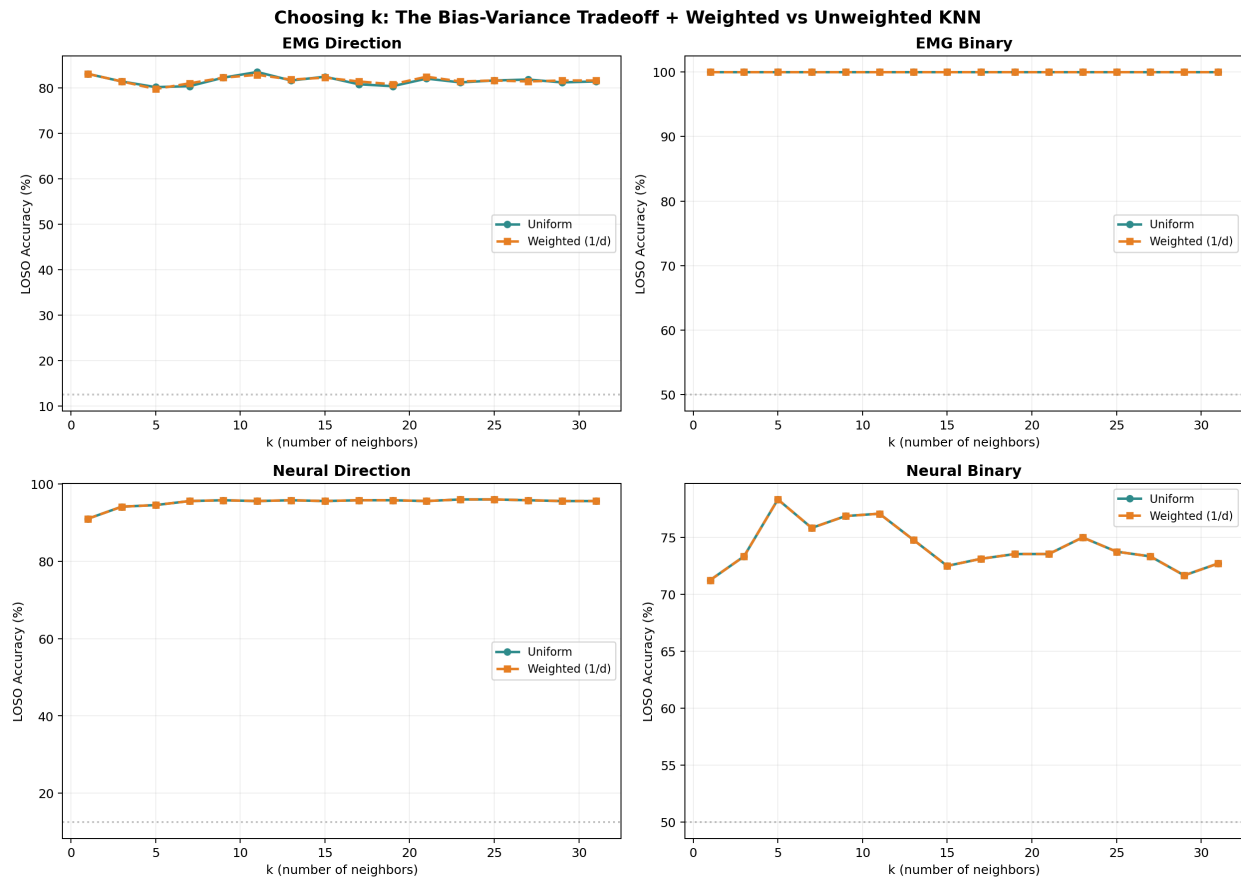


Figure 12. Accuracy vs k for all four tasks. EMG binary is flat at 100% regardless of k . Neural direction peaks around $k = 11$ – 23 . Neural binary peaks at $k = 5$ then declines as more neighbors dilute the subtle diagnostic signal. Red dots mark the optimal k for each task.

Figure 13 illustrates with a hypothetical clinical scenario: classifying patients as mild vs moderate gait impairment using stride length and gait symmetry. The two groups overlap substantially. For a test patient with 11 neighbors (6 mild, 5 moderate), uniform voting picks mild by a slim 6–5 margin. But the *closest* neighbor is moderate (distance = 0.015), and its large weight ($1/d = 67$) flips the weighted vote to moderate. The two boundaries diverge in the overlap zone — precisely where clinical decisions are hardest.

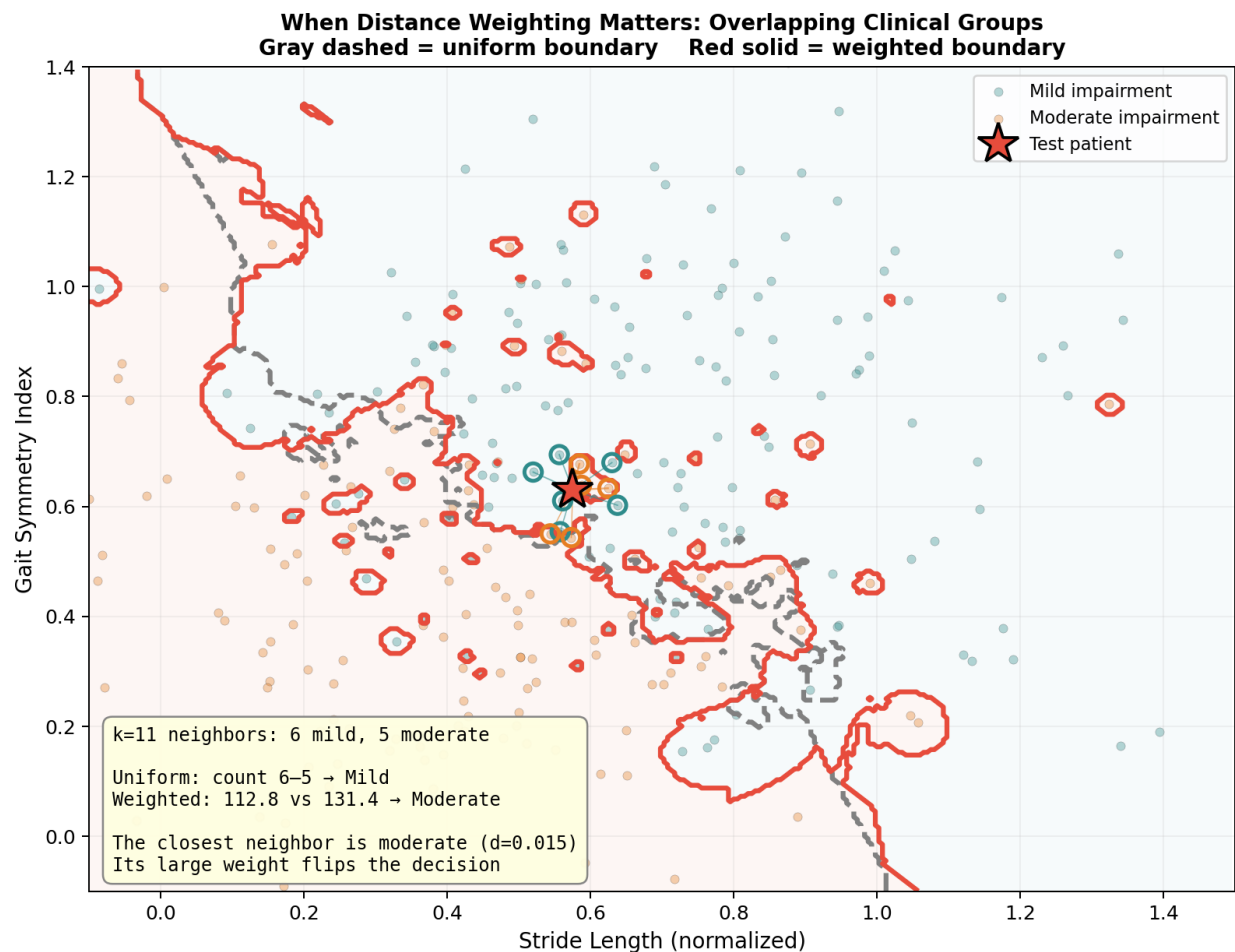


Figure 13. Hypothetical clinical gait data where distance weighting changes the classification. Gray dashed: uniform boundary. Red solid: weighted boundary. The test patient's 11 neighbors include a mix of both classes; the nearest neighbor is moderate and dominates the weighted vote despite being outnumbered 6-5 in the uniform count.

The optimal k depends on both the task and the signal-to-noise ratio. For direction decoding (strong signal, low noise), larger k helps by averaging out trial variability. For clinical diagnosis (subtle signal, high inter-subject variability), smaller k preserves the local structure that distinguishes healthy from impaired. In practice, k is chosen by cross-validation, just like the regularization parameter C in logistic regression.

6. Computational Cost

The train-predict asymmetry

KNN inverts the usual computational profile of machine learning. Parametric models like LR spend time during training (optimizing weights) and are fast at prediction (a single matrix multiplication). KNN is the opposite: training is instant (just store the data), but prediction requires computing the distance from the new trial to every training trial — an $O(n \times d)$ operation, where n is the number of training trials and d is the number of features.

On our dataset (480 trials, 80 neurons), this asymmetry is invisible. KNN predicts a fold in about 2 milliseconds — comparable to Naive Bayes (3 ms) and much faster than LR's training-heavy 29 ms. But the critical difference is **how these times scale**. If we had 10,000 trials and 200 neurons, LR's prediction time stays at $O(d)$ regardless of training size, while KNN's prediction time grows linearly with n .

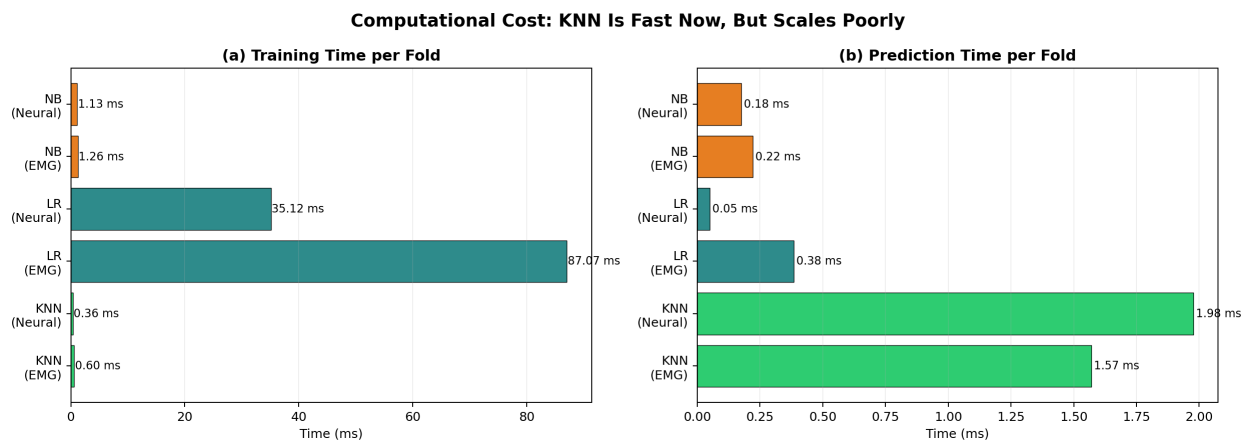


Figure 14. Training and prediction times per LOSO fold. KNN is fast now because our dataset is small. But prediction time scales with training set size ($O(n \times d)$), while parametric models (LR, NB) have $O(d)$ prediction regardless of n .

For motor control applications, this matters most in real-time brain-computer interfaces. A BCI decoder must produce predictions within a few milliseconds to feel responsive. KNN is viable when the training set is small (a few hundred trials from a calibration session), but becomes impractical as training data accumulates across sessions. Parametric models like LR maintain constant prediction speed regardless of how much historical data they were trained on.

There are engineering solutions to speed up KNN — spatial data structures like KD-trees and ball trees reduce the search from $O(n)$ to roughly $O(\log n)$ in low dimensions. But in high dimensions, these structures degrade back to brute-force search, circling back to the curse of dimensionality.

7. Bringing It Together

This week we added our third classification algorithm to the comparison table — and the results reinforce a key theme of this course: **no single algorithm wins everything**. KNN achieves a perfect 100% on EMG binary diagnosis, a result that neither LR (85%) nor NB (65%) could match. Yet on 8-direction decoding, KNN ties or slightly trails the parametric models. The right algorithm depends on the data, the task, and the constraints of the application.

Running Comparison Table Through Week 8

Week	Method	Features	8-Dir	Binary	AUC
5	Logistic Reg (C=10)	Raw EMG (6)	80.8%	85.0%	0.94
6	Logistic Reg (C=10)	Neural (80)	91.2%	73.3%	0.80
6	Population Vector	Neural (80)	95.6%	N/A	N/A
7	Gaussian NB	Raw EMG (6)	81.2%	65.4%	0.68
7	Gaussian NB	Neural (80)	95.2%	77.7%	0.86
7	PCA(6) + NB	Raw EMG (6)	—	85.0%	0.91
8	KNN (k=5)	Raw EMG (6)	80.2%	100.0%	1.00
8	KNN (k=5)	Neural (80)	94.6%	78.3%	0.86
8	KNN (cosine)	Neural (80)	95.8%	—	—

***Figure 15.** Running comparison table through Week 8. KNN achieves perfect EMG binary diagnosis and matches the population vector with cosine distance on neural direction decoding. Bold green highlights the new best result.*

What KNN teaches us about motor control data

- **Subject-level clustering.** The 100% EMG result reveals that healthy and impaired subjects occupy distinct regions of muscle activation space. This is the same synergy structure from Week 4, now visible as a clean separation in 6D. KNN makes this structure actionable: “who does this patient look like?” is a clinically intuitive question.
- **Metric as model.** Cosine distance recovers the population vector without any explicit model of tuning curves. The distance metric is itself a kind of model — it encodes what you believe makes trials “similar.” Euclidean says “amplitude matters.” Cosine says “direction matters.”
- **Dimensionality is a resource and a liability.** More neurons carry more information but also more noise dimensions for KNN to get lost in. The curse of dimensionality is not abstract — we demonstrated a 60-percentage-point accuracy drop from adding irrelevant features.
- **Scalability constrains clinical deployment.** KNN works beautifully on small lab datasets but becomes impractical as training data grows. Parametric models compress the data into fixed-size representations that scale.

We have now seen three classification algorithms: logistic regression (parametric, discriminative), Naive Bayes (parametric, generative), and KNN (non-parametric, instance-based). Each embodies different assumptions about the data. Next week, we

introduce **support vector machines** — a parametric model that uses a kernel trick to operate in a higher-dimensional space, combining the flexibility of KNN's distance-based reasoning with the efficiency of parametric models.